

حلول ورقة العمل 04: المزامنة والدمج

Solutions · Worksheet 04 — Sync & Merge

Pass mark / معيار النجاح /
70 نقطةMandatory evidence / أدلة إلزامية /
8 بنودMarking time / زمن التصحيح المقترح /
8-6 دقائق لكل طالبTotal / التقييم الكلي /
100 نقطة

نسخة المدرب — نماذج الحلول والتقييم / Instructor copy — do not distribute

ما تقيسه هذه الورقة فعلاً What this worksheet really measures

تقيس الورقة الاستقلالية في تشخيص التاريخ. الطالب الذي يعرف أن يسأل: «أين أنا؟ ما الجديد؟ ما الفرق؟ من فعل هذا؟» يستطيع حل 80% من مشكلات الفريق بنفسه. لذلك صُممت معظم المهام لتنتج مخرجات أوامر يعرف الطالب قراءتها — لا مجرد تنفيذها.

EN

This worksheet measures independence in reading history: where am I, what is new, what changed, who did it. The evidence expected is command output the student can actually interpret.

جدول التقييم المُجمَع Marking rubric

# المهمة	النقاط معيار القبول	ملاحظة
1 fetch مقابل pull	15 مخرجات <code>fetch</code> + تفسير <code>behind N</code>	يقيس الفهم لا الأمر
2 الدمج في <code>main</code>	15 دمج ناجح + تحديد نوعه (<code>ff/commit</code>)	الخطأ في النوع = خصم بسيط
3 الاستراتيجيات الثلاث	15 ثلاث مخرجات مختلفة الشكل + تبرير الاختيار سؤال التبرير 5 نقاط	
4 فحص تعديلات الزملاء	15 <code>shortlog</code> / <code>blame</code> مع استنتاج صحيح استنتاج لا نسخ	
5 مختبر التراجع	20 4 حالات منقّدة على الأقل + <code>reflog</code>	الحالة المدمّرة تُشرح لا تُهدّد
6 التنظيف وتقرير الفريق	20 أرقام حقيقية + فرع محذوف بأمان	الأرقام هي الحكم
— الأسئلة (س1-س4)	15 —	يفضّل شفهاً للأضعف

توزيع الدرجة على نوع الدليل Evidence weighting

- مخرجات أمر حقيقية من جهازه = الدرجة الكاملة.
- مخرجات تبدو منسوخة (نفس الـ hashes والأسماء في كل الأوراق) = نصف الدرجة + تنبيه.
- شرح صحيح بلا تنفيذ = ثلث الدرجة (الكفاءة النظرية).
- تنفيذ بلا مخرجات = لا شيء (لا يستطيع إثباته).

Task solutions إجابات المهام

1 fetch مقابل pull

fetch vs. pull

★ 15 نقطة

◆ فهم

الإجابة النموذجية:

```
$ git fetch origin
$ git status -sb
## main...origin/main [behind 2]

$ git log --oneline HEAD..origin/main
f9e8d7c feat(members): add card for leen-docs
3c7d9e1 Merge pull request #7 from nour-dev/feature/nour-card
```

التفسير المطلوب: `fetch` ينزل الالتزامات الجديدة إلى مرجع `origin/main` دون أن يلمس ملفاته أو فرعه المحلي — لذا يظهر `behind 2` (فرعك متأخر بالتزامين). و `pull` = `fetch` + دمج فوري (أو `rebase`)، فتتغير ملفاته مباشرة.

أخطاء متوقعة:

الحالة	الدلالة	الدرجة
نقذ <code>pull</code> فقط ولم يجرب <code>fetch</code>	لم يقس الفرق فعلياً 8 من 15	
لم يفسر <code>behind N</code>	ينقذ بلا فهم	10 من 15
قال « <code>fetch</code> يرفع و <code>pull</code> ينزل»	خلط خطير	3 من 15 + شرح فوري
لا يعرف <code>HEAD..origin/main</code>	أدرجه في الشرح الآن	11 من 15

تشبيه يستوعبه الطالب فوراً *Analogy that lands*

`fetch` = «الاطلاع على بريد الفريق»، `pull` = «قراءة البريد وتنفيذ كل ما فيه فوراً». أسأله: «أيُّهما تفضل صباح يوم تسليم؟» — الجواب الذي تريد سماعه: `fetch` أولاً ثم أفحص ثم أدمج بوعي.

2 الدمج في main

Merging into main

★ 15 نقطة

◆ إتقان

الإجابة النموذجية — الدمج السريع:

```
$ git merge --ff-only origin/main
Updating a1b2c3d..f9e8d7c
Fast-forward
 data/members/leen-docs.json | 9 ++++++++
 2 files changed, 13 insertions(+), 4 deletions(-)
```

الإجابة النموذجية — الدمج بعقدة:

```
$ git merge feature/omar-qa-card
Merge made by the 'ort' strategy.
 data/members/omar-qa.json | 9 ++++++++
 1 file changed, 9 insertions(+)
 create mode 100644 data/members/omar-qa.json
```

معيار القبول: الدمج حدث ونجح، والطالب يعرف أي نوع وقع ولماذا:

نوع الدمج	متى يقع	كيف يعرفه
Fast-forward	فرعه المحلي لم يتقدّم	Fast-forward ... Updating وبلا عقدة
Merge commit	الفرعان تقدّما (عمل متبادل)	Merge made by the 'ort' strategy. + عقدة في <code>--graph</code>
Already up to date	لا جديد	Already up to date.

أخطاء متوقعة: تنفيذ `git pull` مع تعديلات محلية غير محفوظة ⇒

`error: Your local changes would be overwritten by merge` ← الدرس `git stash`. من رفع على `main`
بدل الدمج المحلي ⇒ خصم 5 نقاط + مراجعة إعدادات حماية الفرع مع المدّرب.

3 الاستراتيجيات الثلاث

The three strategies

★ 15 نقطة

◆ إتقان

الإجابة النموذجية (ملخص المخرجات المتوقعة):

```
# 1) merge commit
* 5b6c7d8 (HEAD → main) Merge pull request #6 from ali-h/feature/ali-card
|\
| * 1a2b3c4 feat(members): add card for ali-h
|/

# 2) squash
$ git merge --squash feature/sara-fixes
Squash commit -- not updating HEAD
$ git commit -m "feat(ui): improve member card layout (#9)"
7a8b9c0 (HEAD → main) feat(ui): improve member card layout (#9)

# 3) rebase
$ git rebase main
Successfully rebased and updated refs/heads/feature/nour-card.
* 4d5e6f7 (HEAD → feature/nour-card) feat(members): add card for nour-dev
* 5b6c7d8 Merge pull request #6 from ali-h/feature/ali-card
```

سؤال التبرير (5 نقاط): الإجابة الممتازة على «أي استراتيجية تفضل لفريق فيه 20 طالباً يضيفون بطاقات؟» = **Squash and merge**، لأن كل بطاقة تصبح التزاماً واحداً نظيفاً على `main`، فيبقى تاريخ المشروع قابلاً للقراءة مع 20 مساهماً، ويُربط كل التزام برقم طلب السحب (12). والإجابة المقبولة أيضاً: Merge commit مع تبرير «نريد رؤية كل خطوة»، بشرط ذكر ثمنها: تاريخ مزدحم يُعقد.

أخطاء متوقعة: - من كتب `git merge --squash` ثم نسي `git commit` ← يقول «لم يحدث شيء»؛ هذا هو الصواب التقني لكنه ينتظر خطوة يدوية → وجهه وأعطه 4 من 5. - من نغذ `rebase` على فرع مشترك ← 5 - ومناقشة مخاطر إعادة كتابة التاريخ. هذا الدرس سيكون محورياً في الورقة 05. - تبرير بلا سبب («لأنها الأفضل») ⇒ نصف نقاط السؤال.

4 فحص تعديلات الزملاء

Inspecting teammates' changes

★ 15 نقطة

◆ تحقيق

الإجابات النموذجية:

السؤال

من عدّل

الجواب الصحيح: **عدّله الجميع** — يظهر في كل التزام لكل عضو (وهذا هو جوهر درس الملف المُولّد)

آخر `data/members-index.json` مرة؟

يُقرأ من `git shortlog -sn --all` — يقبل الاختلاف حسب توقيت الجلسة، المهم أن يكون من مخرجات فعلية

عدد الالتزامات لكل زميل

السؤال

الجواب

هل `main` متساوٍ مع `origin/main` ؟ يعتمد على الحالة لحظة التسليم – المهم أن يعرف كيف تحقق أدلة متوقعة:

```
$ git blame data/members/leen-docs.json
f9e8d7c9 (leen-docs 2026-10-06 11:02:40 +0300 1) {
f9e8d7c9 (leen-docs 2026-10-06 11:02:40 +0300 2)   "fullName": "لينا حدّاد",
```

الاستنتاج الذي تريد رؤيته (سطر واحد): «ملف البطاقة يخص شخصاً واحداً وهو مؤلفه فقط، أما ملف الفهرس فهو مشترك ويتغير من الجميع» – هذا الاستنتاج هو الذي يمنح الدرجة الكاملة.

أخطاء متوقعة: من كتب أن `index.json` «يجب أن يحلّه آخر من عدّله» ← مفهوم ناقص، نَبّهه أن الصواب إعادة التوليد. من نسي `--all` فلم يرَ فروع الزملاء ⇒ -3.

5 مختبر التراجع

Undo lab

★ 20 نقطة

◆ إتقان

معيار القبول: 4 حالات منقّذة فعلياً على الأقل من هذه الحالات:

النقاط	المخرج المتوقع	الأمر الصحيح	الحالة
4	فارغ <code>git status -s</code>	<code>git restore <file></code>	تعديل غير مُلتزم
4	يعود إلى M <code>git restore --staged <file></code>		ملف مُمرّر خطأً
4	hash جديد + رسالة صحيحة <code>git commit --amend -m "..."</code>		رسالة آخر التزام خاطئة
4	التزام "..." Revert جديد <code>git revert <sha></code>		التزام مرفوع خاطئ
4	ثم عودة التعديل <code>stash list</code> <code>git stash pop</code> ثم <code>git stash</code>		تبديل فرع وعمل ناقص

الإجابة النموذجية للسؤال التحليلي (حذف زميل ملفاً وتزمره ورفعها): لا يجوز `reset --hard` ولا `revert` لالتزام زميل قد يكون بُني عليه عمل. الطريقة السليمة:

```
$ git show <sha> --stat # تأكد من الالتزام الذي حذف الملف
$ git checkout <sha>^ -- data/members/x.json # استرجع الملف من الالتزام السابق للحذف
$ git add data/members/x.json
$ git commit -m "fix(members): restore deleted card for x"
```

أو `git revert <sha> --no-edit` إن كان الالتزام لم يُبنَ عليه شيء. التفريق بين الحالتين هو معيار الدرجة الكاملة.

أخطاء متوقعة ووزنها:

الخطأ	الدلالة	الدرجة
جرب <code>reset --hard</code> ثم قال «ضاع عملي» تعليمي ومتوقع (إن كان على نسخة جديدة) لا خصم إن كان على مستودع التجربة	استخدم <code>git checkout</code> لحذف التعديلات أمر قديم/خطير	من 4 + شرح <code>restore</code>
لم يعرف <code>reflog</code>	أهم شبكة أمان	0 في سؤال التحليل + تدريب فوري
حذف فرع زميل بـ <code>-D</code>	سلوك مرفوض	-5 وإرشاد مباشر

تمرين دقيقتين لإحياء الصدمة التربوية A two-minute demonstration

الطلب من طالب تجربة `git reset --hard` ثم استرجاع الحالة بـ `git reflog` هو أقوى درس عملي في هذه الورقة. اجعله يعرضه على المجموعة: يذكّرهم أن Git لا ينسى بسهولة — لكن `reflog` سجل محلي لا يُرفع إلى GitHub، فلن ينقذ زميلاً على جهاز آخر.

6 التنظيف وتقرير الفريق

Cleanup and team report

★ 20 نقطة

◆ إتقان

الإجابة النموذجية:

```
$ git branch --merged main
feature/ali-card
feature/leen-card
* main

$ git branch -d feature/ali-card feature/leen-card
Deleted branch feature/ali-card (was 1a2b3c4).

$ git fetch --prune
- [deleted]          (none)      → origin/feature/ali-card

$ git shortlog -sn --all
11  omar-qa
9   sara-design
8   leen-docs
5   nour-dev
4   ali-h
```

معيار القبول للتقرير: أن تكون الأرقام قابلة للتفسير ومتسقة داخلياً (مثال: عدد البطاقات = عدد ملفات `data/members/*.json` ناقص قالب والمثال والمدرب)، وأن يذكر الطالب استنتاجاً واحداً تعلّمه من التاريخ، لا مجرد أرقام منسوخة.

الإجابة النموذجية على السؤال التحليلي (لو رفع الجميع على main مباشرة): «سيصبح `main` غير مستقر: أي خطأ في أي بطاقة يظهر فوراً لكل الفريق؛ ولا يمكن مراجعة أي تعديل قبل دخوله؛ وسيعارض الطلاب عند كل رفع في ملف الفهرس ويضيع عمل من يخسر السباق، بلا سجل يبيّن من أفسد المشروع.»

معيار الدرجة: نقطتان للفكرة الأمنية (عدم استقرار)، نقطتان للفكرة التنظيمية (المراجعة)، نقطة لتسمية الملف الحساس في المشروع (`members-index.json`).

تقييم الفهم الحقيقي Measuring real understanding

صّف الطلاب في هذه الورقة على ثلاثة مستويات: **المستوى أ** — ينفذ ويعرف لماذا (ينتقل للورقة 05 كمستقل). **المستوى ب** — ينفذ بالأوامر لكن لا يقرأ المخرجات (يحتاج مرافقة في التعارضات). **المستوى ج** — يحفظ الأوامر بلا فهم (سيحتاج مساعدة فردية في الورقة 05 — رتبه مع طالب من المستوى أ كزوج تدريبي).

س1 fetch مقابل pull

fetch vs pull

★ 4 نقطة

◆ فهم

الجواب: fetch يسحب التحديثات فقط إلى `origin/<branch>` دون تغيير الملفات — آمن لأنه يسمح بالفحص قبل الدمج. `pull` = `fetch` + دمج (أو rebase)، فتتغير الملفات ويحتمل التعارض فوراً. أستعمل `fetch` قبل أي دمج مهم أو قبل المظاهرة/التسليم.

س2 متى Squash ومتى Merge

Squash vs merge

★ 4 نقطة

◆ تحليل

الجواب الممتاز: Squash عندما تكون التزامات الفرع صغيرة ومتكررة عن مهمة واحدة (مثل بطاقة واحدة أضيفت ثم صُححت ثلاث مرات) — فنحفظ المشروع نظيفاً. Merge commit عندما يكون الفرع طويلاً يحمل مهام متعددة وُهِمنا حفظ تدرّجها للحق. **مثال من مشروعنا:** Squash لوظيفة `fix tag typo`، Merge لفرع الميزة الكبير `feature/search-filter`.

س3 لماذا لا rebase لفرع مشترك

No rebase on shared branches

★ 4 نقطة

◆ تحليل

لأن rebase يعيد كتابة التاريخ (ينشئ التزامات جديدة بنفس المحتوى و hashes مختلفة)؛ فيصبح تاريخ زملائك مرتبطاً بـتزامات لم تعد موجودة، وتحديث فوضى دفع ورفض وإعادة سحب متكررة. البديل: `git merge`، وإن أردت تاريخاً خطئياً فاستخدم `rebase` على فرعك الشخصي فقط ثم `push --force-with-lease`.

س4 تعديل خاطئ على main

Bad change on main

★ 3 نقطة

◆ تطبيق

الجواب: `git revert <sha>` (مع `--no-edit` للتسريع) لإنشاء التزام يعكس الخاطئ بلا حذف تاريخ أحد. وإن كان الالتزام آخر التزام ولم يبن عليه أحد شيئاً بعد، يمكن `git reset --soft HEAD~1` ثم تصحيح ثم `push --force-with-lease` — لكن فقط بعد التنسيق مع الفريق وتأكيد عدم وجود عمل مبني عليه.

ملخص التصحيح Marking summary

سجل الدرجات/ Grade sheet

الاسم م1 15/ م2 15/ م3 15/ م4 15/ م5 20/ م6 20/ أسئلة 15/ المجموع 100/ المستوى (أ/ب/ج)

قبل الانتقال، اكتب لكل طالب في المستوى ب/ج مهمة محددة: «تدريب `reflog`»، «قراءة مخرجات `git status` كل مرة بصوت مسموع»، «إعادة تجربة `stash`». الورقة 05 تُنتج تعارضاً مقصوداً — والطالب الذي لم يفهم `stash` سيجمّد الفريق عند أول `pull`.